

The Foundation - System Mission, Ubuntu, Terminal, and GitHub

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 8, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

- 1 Mission and Foundations
- 2 Understanding Ubuntu and the Linux Environment

1 Mission and Foundations

2 Understanding Ubuntu and the Linux Environment

First, Observe the Destination



The Internship Mission

Build a reproducible autonomous-drone inspection system in simulation

Engineering Need

Inspection tasks may involve large, repetitive, difficult-to-access, or potentially hazardous areas. Aerial robots can help collect visual evidence efficiently and consistently.

The Internship Mission

Build a reproducible autonomous-drone inspection system in simulation

Engineering Need

Inspection tasks may involve large, repetitive, difficult-to-access, or potentially hazardous areas. Aerial robots can help collect visual evidence efficiently and consistently.

Internship Mission

Configure, program, simulate, test, and document a quadcopter that:

- ▶ follows a planned inspection mission;
- ▶ observes a simulated scene with a camera;
- ▶ detects selected objects;
- ▶ records useful mission evidence.

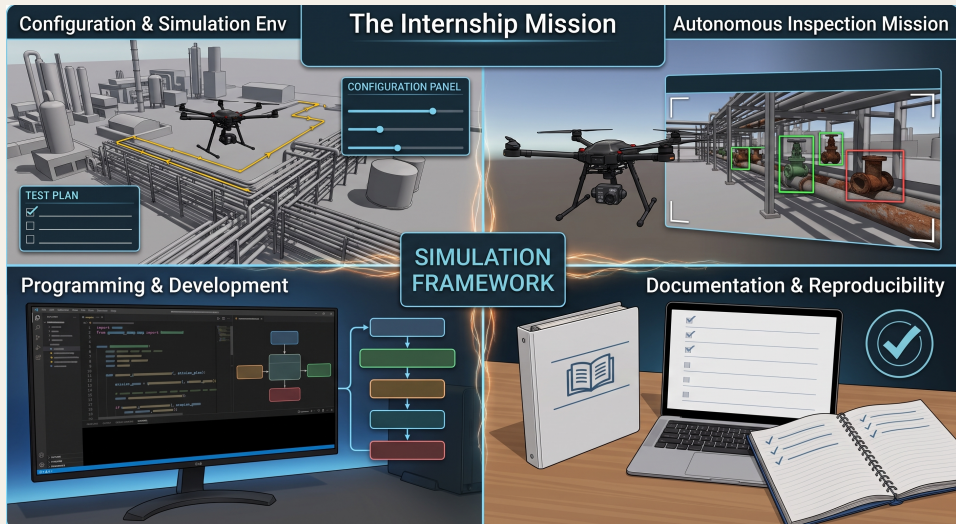
Observe the complete demonstration before studying its components.

Gazebo world → PX4-controlled flight → ROS 2 data exchange → Camera perception → Mission evidence

Important Boundary

A successful simulation is evidence of software integration, not proof that a real aircraft is ready or safe to fly.

First, Observe the Destination



The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → **Simulation** → **Flight control** → **Communication** → **Perception** → **Evidence**

The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → Simulation → Flight control → Communication → Perception → Evidence

A Chain Is Verified Link by Link

A detector cannot process a missing camera stream. A mission node cannot guide the vehicle through an incorrect interface. A working local setup cannot be reproduced without controlled dependencies and documentation.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

Prepare

Configure the computer and create a reproducible project environment.

Build

Program, simulate, connect, and test the robotic components.

Deliver

Document the work, preserve the results, and share a working project.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

Prepare

Configure the computer and create a reproducible project environment.

Build

Program, simulate, connect, and test the robotic components.

Deliver

Document the work, preserve the results, and share a working project.

Think of the Project as a Team

Ubuntu provides the workplace, Bash executes instructions, Git preserves progress, Python implements logic, ROS 2 connects components, Gazebo provides the simulated world, and AI interprets camera data.

You are not expected to master everything on Day 1.

You will learn each foundation when the project needs it.

Technology Ecosystem I

Preparing the environment and preserving your work

Ubuntu: The Workplace

Ubuntu is the Linux distribution used to run the project. It provides the files, processes, packages, and services required by the robotics tools.

Terminal and Bash: The Control Desk

The **terminal** is the window where commands are entered. **Bash** is the shell that reads and executes those commands, variables, and scripts.

Git: The Project Timeline

Git records changes on the computer. It allows students to create checkpoints, compare versions, and recover a previous working state.

GitHub: The Shared Project Home

GitHub hosts the Git repository online. It supports collaboration, documentation, issue tracking, review, and delivery of the internship project.

Technology Ecosystem I

Preparing the environment and preserving your work

Ubuntu: The Workplace

Ubuntu is the Linux distribution used to run the project. It provides the files, processes, packages, and services required by the robotics tools.

Terminal and Bash: The Control Desk

The **terminal** is the window where commands are entered. **Bash** is the shell that reads and executes those commands, variables, and scripts.

Git: The Project Timeline

Git records changes on the computer. It allows students to create checkpoints, compare versions, and recover a previous working state.

GitHub: The Shared Project Home

GitHub hosts the Git repository online. It supports collaboration, documentation, issue tracking, review, and delivery of the internship project.

How These Foundations Support the Drone Project

Ubuntu runs the tools → **Bash** launches the workflow → **Git** preserves the work → **GitHub** makes the project reproducible and shareable.

Technology Ecosystem II

From programming logic to simulated drone intelligence

Python: The Logic

Implements readable robotics programs, data processing, mission utilities, tests, and AI integration.

ROS 2: The Communication Network

Connects software components using nodes, topics, messages, services, and parameters. ROS 2 is not an operating system.

Gazebo: The Virtual Test Environment

Simulates the drone, camera, objects, environment, and physics before any real-world deployment.

PX4: The Flight Controller

Estimates the drone state, stabilizes the vehicle, and follows flight references inside the simulation.

AI and YOLOv8: The Perception

Processes camera images and detects objects of interest. Detection results provide observations, not direct flight control.

Important Engineering Boundary

A successful simulation helps us develop and test the system, but it does not guarantee that the drone is ready for safe real-world flight.

Internship Deliverables

What students are expected to build and submit

Working Engineering Assets

- ▶ structured Git repository and clear history;
- ▶ reproducible environment and configuration;
- ▶ ROS 2 packages, Python source code, and launch files;
- ▶ working Gazebo/PX4 inspection simulation;
- ▶ camera and object-detection integration.

Evidence and Communication

- ▶ layer-level and end-to-end tests;
- ▶ logs, screenshots, and mission results;
- ▶ installation and execution instructions;
- ▶ final demonstration;
- ▶ technical report with results, limitations, and future work.

Delivery Standard

A reviewer should be able to clone the repository, follow the documentation, reproduce the demonstration, and understand what was verified.

Learning Roadmap

Each foundation unlocks one part of the final GitHub project

Foundation	Contribution to the Project	Practical Outcome
Ubuntu	Provides the Linux engineering environment	Prepare the workstation
Terminal and Bash	Operate, configure, launch, inspect, and troubleshoot	Run the documented workflow
Git and GitHub	Preserve and share a reproducible project	Clone, manage, document, and deliver
Python for robotics	Express logic and process project data	Read and modify project programs
ROS 2	Connect software components through defined interfaces	Build and verify communication
Gazebo and PX4	Simulate the world and controlled drone motion	Execute the inspection mission
AI integration	Convert camera data into perception results	Produce inspection evidence

Goal of the Tutorials

End-to-End Objective

The tutorials are not isolated courses. Together, they provide the foundations required to reproduce, understand, test, and improve the autonomous-drone project hosted on GitHub.

Progress from guided reproduction to engineering ownership

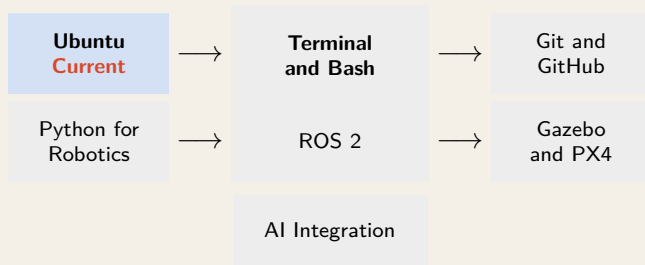
Progress from guided reproduction to engineering ownership

1. **Orient:** understand the mission, architecture, repository, and expected evidence.
2. **Prepare:** configure Ubuntu, tools, dependencies, workspace, and project files.
3. **Learn the foundations:** practise Bash, Git, Python, ROS 2, Gazebo, PX4, and perception concepts in context.
4. **Integrate:** connect mission logic, simulated flight, camera data, and object detection.
5. **Verify:** test one layer at a time before testing the complete pipeline.
6. **Deliver:** publish reproducible code, configuration, evidence, documentation, and a technical report.

Transition to the Technical Foundations

One foundation completed, the next one begins

Internship Learning Roadmap



1 Mission and Foundations

2 Understanding Ubuntu and the Linux Environment

What Is an Operating System?

The coordinator between the computer, applications, and user

User and applications $\longleftrightarrow$ **Operating system** $\longleftrightarrow$ **Hardware**

What Is an Operating System?

The coordinator between the computer, applications, and user

User and applications $\longleftrightarrow$ **Operating system** $\longleftrightarrow$ **Hardware**

Intuitive Analogy

Think of the operating system as the manager of a laboratory. It assigns resources, organizes equipment, and controls access.

What It Manages

- ▶ files and storage;
- ▶ memory and running programs;
- ▶ users and permissions;
- ▶ devices, networks, and services.

Project Connection

When Gazebo uses the GPU, PX4 runs as a process, or ROS 2 reads a camera, the operating system coordinates the required resources.

Who Manages the Computer?

Programs need controlled access to physical resources

Active Programs (The Demand)

During the drone project, multiple heavy programs run simultaneously:

- ▶ **Gazebo**: simulates physics;
- ▶ **PX4**: controls flight logic;
- ▶ **ROS 2**: exchanges messages;
- ▶ **YOLOv8**: detects objects.

Shared Resources (The Supply)

All these programs must compete for the same physical hardware:

- ▶ **Processor** (CPU time);
- ▶ **Memory** (RAM allocation);
- ▶ **Files** (Storage access);
- ▶ **Devices** (GPU and Network).

The Fundamental Problem & Key Intuition

These programs cannot safely share resources independently. If Gazebo and YOLOv8 both demand 100% of the GPU at the same time, the system crashes.

Solution: Between the software and the physical computer, we need a trusted, central traffic controller. This low-level manager is the **kernel**.

The Kernel

The low-level manager between software and hardware

Applications and Robotics Programs

Gazebo PX4 ROS 2 Python YOLOv8



Linux Kernel

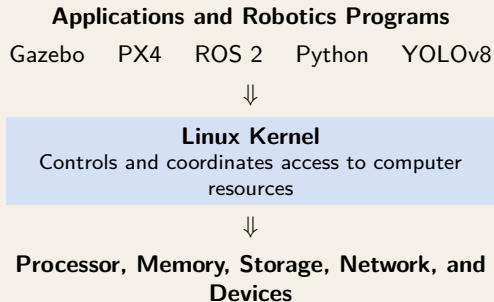
Controls and coordinates access to computer resources



**Processor, Memory, Storage, Network, and
Devices**

The Kernel

The low-level manager between software and hardware



What Does the Kernel Manage?

- ▶ **Processes:** which programs are running and when they use the processor;
- ▶ **Memory:** which memory regions belong to each running program;
- ▶ **Files and storage:** how data is read, written, and protected;
- ▶ **Devices:** how software communicates with hardware through drivers;

Linux Is More Than a Kernel

From the technical core to a complete operating environment

1. The Linux Kernel

The low-level engine that talks directly to the hardware:

- ▶ processes;
- ▶ memory;
- ▶ files & networks;
- ▶ hardware devices.

2. User Tools & Ecosystem

Software that makes the kernel actually usable by humans:

- ▶ a shell (Bash);
- ▶ terminal commands;
- ▶ software libraries;
- ▶ a package manager (apt).

3. Linux Distribution

A complete, ready-to-use
operating system assembled
from both parts.

Ubuntu 22.04

Linux Is More Than a Kernel

From the technical core to a complete operating environment

1. The Linux Kernel

The low-level engine that talks directly to the hardware:

- ▶ processes;
- ▶ memory;
- ▶ files & networks;
- ▶ hardware devices.

2. User Tools & Ecosystem

Software that makes the kernel actually usable by humans:

- ▶ a shell (Bash);
- ▶ terminal commands;
- ▶ software libraries;
- ▶ a package manager (apt).

3. Linux Distribution

A complete, ready-to-use operating system assembled from both parts.

Ubuntu 22.04

The Operating System Equation

Kernel + Tools, Shell & Apps = Distribution (Ubuntu)

Linux Is More Than a Kernel

From the technical core to a complete operating environment

The Operating System Equation

Kernel + Tools, Shell & Apps = Distribution (Ubuntu)



Operating System Comparison

Why don't we just program the drone on Windows or macOS?

Ubuntu (Linux)

The Robotics Standard

- ▶ **Native:** ROS 2 and Gazebo are built primarily for this OS.
- ▶ **Open:** Complete, unfiltered access to hardware.
- ▶ **Focus:** Command-line automation.

Windows

The General Desktop

- ▶ **Familiar:** Broadest consumer support.
- ▶ **Focus:** Graphical User Interfaces (GUI).
- ▶ **Barrier:** Requires Virtual Machines or WSL to run Linux tools, adding overhead.

macOS

The Premium Workspace

- ▶ **Unix-based:** Excellent developer environment.
- ▶ **Closed:** Locked to Apple hardware.
- ▶ **Barrier:** Compiling complex robotics binaries natively is often unsupported.

Operating System Comparison

Why don't we just program the drone on Windows or macOS?

Ubuntu (Linux)

The Robotics Standard

- ▶ **Native:** ROS 2 and Gazebo are built primarily for this OS.
- ▶ **Open:** Complete, unfiltered access to hardware.
- ▶ **Focus:** Command-line automation.

Windows

The General Desktop

- ▶ **Familiar:** Broadest consumer support.
- ▶ **Focus:** Graphical User Interfaces (GUI).
- ▶ **Barrier:** Requires Virtual Machines or WSL to run Linux tools, adding overhead.

macOS

The Premium Workspace

- ▶ **Unix-based:** Excellent developer environment.
- ▶ **Closed:** Locked to Apple hardware.
- ▶ **Barrier:** Compiling complex robotics binaries natively is often unsupported.

The Engineering Decision

While Windows and macOS are excellent for daily tasks, forcing them to run advanced robotics software requires workarounds (like Virtual Machines) that drain CPU and RAM. **Ubuntu provides the native, zero-overhead environment required for the drone's digital twin.**

Why One Exact Ubuntu Version Matters

Compatibility is part of engineering, not an installation detail

Ubuntu version → available packages → compatible robotics stack

Why One Exact Ubuntu Version Matters

Compatibility is part of engineering, not an installation detail

Ubuntu version → available packages → compatible robotics stack

Example Compatibility Pairs

- ▶ ROS 2 Humble: Ubuntu 22.04
- ▶ ROS 2 Jazzy: Ubuntu 24.04

Internship Rule

Follow the version declared in the project documentation. Do not replace it only because another release is newer.

Why This Matters

A tutorial written for another Ubuntu or ROS 2 release may contain different package names, dependencies, or setup commands.

The Filesystem: One Organized Tree

Linux organizes files from a single starting point

`/ → /home → /home/student → project workspace`

The Filesystem: One Organized Tree

Linux organizes files from a single starting point

`/ → /home → /home/student → project workspace`

Filesystem

The rule and structure used to organize files and directories. The top of the Linux tree is called **root** and is written as `/`.

Home Directory

Each user normally works inside a personal directory such as `/home/student`. The shortcut `~` means “my home directory.”

Project Connection

File paths tell scripts where to find worlds, models, source code, configuration files, logs, and datasets.

Users, Administrator, and Root

Permissions protect the system from accidental changes

User

A normal account for daily work, coding, simulation, and documents.

Administrator

A trusted user allowed to perform selected system-level tasks.

Root

The system's all-powerful administrative account. Root can modify almost anything.

Users, Administrator, and Root

Permissions protect the system from accidental changes

User

A normal account for daily work, coding, simulation, and documents.

Administrator

A trusted user allowed to perform selected system-level tasks.

Root

The system's all-powerful administrative account. Root can modify almost anything.

What sudo Means

sudo requests permission to run one command with administrative privileges. It is not a decoration to add when a command fails.

Safe Working Habit

Work as a normal user. Use administrative privileges only when the command is understood and the trusted documentation requires them.

Packages, Dependencies, and Repositories

How Ubuntu obtains and maintains software

Package

A prepared unit of software with metadata needed for installation and maintenance.

Dependency

Another package or library required for the software to work.

Repository

A configured software source from which packages and updates can be obtained.

Terminal, Shell, and Bash

The window, the interpreter, and the specific tool

Terminal

The application window that displays text and accepts keyboard input.

Shell

The command interpreter that reads instructions and starts programs.

Bash

A widely used shell that supports commands, variables, scripts, redirection, and automation.

Terminal, Shell, and Bash

The window, the interpreter, and the specific tool

Terminal

The application window that displays text and accepts keyboard input.

Shell

The command interpreter that reads instructions and starts programs.

Bash

A widely used shell that supports commands, variables, scripts, redirection, and automation.

Restaurant Analogy

The terminal is the counter, the shell is the person interpreting the order, and Bash is one specific language and rule set used to express that order.

Next tutorial: we will use the terminal and Bash safely, one command at a time.

Open Source Is More Than Free Access

It is a collaborative engineering model

Open-Source Software

Source code is available under a license that defines how it may be inspected, used, modified, and shared.

Modular Software

Complex systems are divided into components with clear responsibilities and interfaces.

Community Collaboration

Developers contribute code, reviews, documentation, tests, bug reports, and technical discussions.

Documentation

Installation steps, assumptions, versions, and known problems are recorded so that others can repeat the work.

Professional Habit

Reusing open-source software does not remove engineering responsibility. We still verify versions, licenses, interfaces, behavior, and assumptions.

Reproducibility: “It Works on My Computer” Is Not Enough

Another student should be able to obtain the same result

Same repository + same versions + documented steps → comparable result

Reproducibility: “It Works on My Computer” Is Not Enough

Another student should be able to obtain the same result

Same repository + same versions + documented steps → comparable result

Record

- ▶ Ubuntu and ROS 2 versions;
- ▶ required packages and dependencies;
- ▶ configuration and environment variables;
- ▶ launch and test commands.

Verify

- ▶ start from a clean terminal;
- ▶ follow the documented sequence;
- ▶ compare expected and observed outputs;
- ▶ report missing assumptions.

Error Messages Are Engineering Evidence

Troubleshooting begins with observation, not random commands

1. **Stop and read** the complete message.
2. **Locate** the first useful error, file, package, or process name.
3. **Check context:** directory, command, versions, and environment.
4. **Form one hypothesis** and test one change at a time.
5. **Document the verified fix** and how it was validated.

Error Messages Are Engineering Evidence

Troubleshooting begins with observation, not random commands

1. **Stop and read** the complete message.
2. **Locate** the first useful error, file, package, or process name.
3. **Check context**: directory, command, versions, and environment.
4. **Form one hypothesis** and test one change at a time.
5. **Document the verified fix** and how it was validated.

Beginner-Friendly Question

Instead of asking “Why does nothing work?”, ask: “Which component failed, what evidence identifies it, and what changed since the last working state?”

Avoid

Blindly adding sudo, repeatedly reinstalling everything, or copying commands written for another software version.

What You Should Now Be Able to Explain

Conceptual checkpoint before using the command line

- ▶ An **operating system** coordinates applications, users, and hardware.
- ▶ **Ubuntu** is a Linux distribution; **Linux** provides the kernel foundation.
- ▶ Ubuntu is selected because it offers a supported and reproducible robotics environment.
- ▶ A **filesystem** organizes files; users and permissions control access.
- ▶ Packages come from repositories and may require dependencies.
- ▶ Running programs are processes that receive settings from their environment.
- ▶ Open-source robotics depends on collaboration, modularity, documentation, reproducibility, and disciplined troubleshooting.

What You Should Now Be Able to Explain

Conceptual checkpoint before using the command line

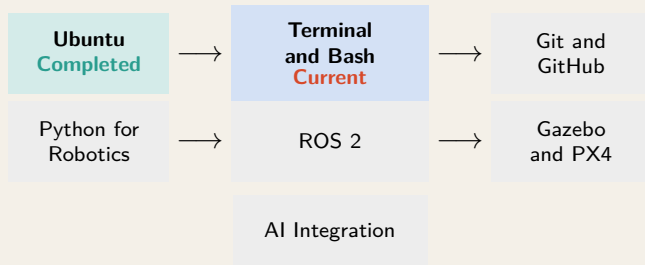
- ▶ An **operating system** coordinates applications, users, and hardware.
- ▶ **Ubuntu** is a Linux distribution; **Linux** provides the kernel foundation.
- ▶ Ubuntu is selected because it offers a supported and reproducible robotics environment.
- ▶ A **filesystem** organizes files; users and permissions control access.
- ▶ Packages come from repositories and may require dependencies.
- ▶ Running programs are processes that receive settings from their environment.
- ▶ Open-source robotics depends on collaboration, modularity, documentation, reproducibility, and disciplined troubleshooting.

Next: Terminal and Bash Foundations

Transition to the Technical Foundations

One foundation completed, the next one begins

Internship Learning Roadmap



References and Further Reading I

Ubuntu and Linux foundations



Ubuntu, *About the Ubuntu Project*.

<https://ubuntu.com/about>



Canonical, *Official Ubuntu Documentation*.

<https://help.ubuntu.com/>



Canonical, *Ubuntu Desktop Documentation*.

<https://documentation.ubuntu.com/desktop/>



The Linux Kernel Organization, *Linux Kernel Documentation*.

<https://docs.kernel.org/>



Linux Foundation, *Filesystem Hierarchy Standard*.

<https://refspecs.linuxfoundation.org/fhs.shtml>

References and Further Reading II

Packages, permissions, and robotics compatibility



Ubuntu, *Ubuntu Packages Search*.

<https://packages.ubuntu.com/>



Debian Project, *APT User's Guide*.

<https://www.debian.org/doc/manuals/apt-guide/>



Ubuntu Community Help Wiki, *RootSudo*.

<https://help.ubuntu.com/community/RootSudo>